



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

25CY201 -APPLIED STATISTICS USING PYTHON

ACADEMIC YEAR 2025-2026

PRACTICAL RECORD

Name : SAKTHI .R

Reg.No : 25CS139

Branch & Name :B.E. COMPUTER SCIENCE AND ENGINEERING

Class/Section : I B.E.CSE B

Year/Sem :I/II



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade

Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

25CY201 -APPLIED STATISTICS USING PYTHON

PRACTICALRECORD

Name : _____ Reg.no: _____

Class: _____ Branch: _____

BONAFIDE CERTIFICATE

Certified bonafide record of work done by Mr./ Ms.

Reg.No.....during the academic year 2025 – 2026(EVEN SEM)

Faculty Incharge

HoD/CSE

Submitted for End Semester practical Examination, held on.....

INTERNAL EXAMINER

EXTERNAL EXAMINER



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade

Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

INDEX

EX.N O	DATE	NAME OF THE PROGRAM	MARKS	SIGNATURE
-----------	------	---------------------	-------	-----------

1		Execution of matrix operations using NumPy to perform in Data analysis.		
2		Computation of mean, variance and standard deviation using Python.		
3		Data visualization for univariate and bivariate datasets.		
4		Correlation and regression analysis using Python.		
5		Hypothesis testing using t-test and chisquare test		
6		Analysis of variance - based comparison of datasets.		
7		Principal Component Analysis using Python.		
8		Application of K-Means clustering for engineering dataset		
9		Time-series trend analysis		
10		Mini project: Statistical analysis of realworld data.		



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade

Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

25CY201 -APPLIED STATISTICS USING PYTHON

EVEN SEMESTER 2025-2026 Name

of the lab In-charge:

CONTINUOUS EVALUATION SHEET RUBRICS TABLE

Criteria	Range of Marks			
	Excellent	Good	Average	Below Average
Objective(20)	18-20	14-17	10-13	0-9
Coding(30)	27-30	21-26	15-20	0-14
Compilation and Execution(30)	27-30	21-26	15-20	0-14
Documentation(10)	9-10	7-8	5-6	0-4
Viva(10)	9-10	7-8	5-6	0-4

NAME OF THE LABORATORY:			25CY201 -APPLIED STATISTICS USING PYTHON			
YEAR, SEMESTER AND BRANCH:			I YEAR / II SEMESTER / B.E / B.TECH - COMMON TO CSE, IT, AI&DS, CSD, CYBER			
NAME OF THE FACULTY MEMBER:						
Sl. No	Criteria	Marks	Excellent	Good	Average	Poor
			91% - 100%	71% - 90%	50% - 70%	<50%
1	Objective	20	Clear understanding of the objectives with all details and explain very well(4.55-5)	Clear understanding of the objectives but not able to give clear explanation(3.554.5)	Moder	
2	Coding	30	Able to list all commands and give relevant explanation to them (13.65 - 15)	Able to list most commands and give explanation to some extend (10.65 - 13.5)	Able to list popular commands and explain partially(7.510.5)	Not able to list any commands and no explanation (<7.5)
3	Compilation and Execution	30	Materials and Methods are identified properly (54.6 - 60)	Materials are mentioned without methods (30.6 - 54)	Not able to give correct material and involves many mistakes (30 - 42)	Not able to recognize the materials and methods to proceed (<30)
4	Documentation	10	Printouts and documentation fully completed (9.1 - 10)	Printouts and documentation mostly completed (7.1 - 9)	Printouts and partially Completed documentation (5 - 7)	No printouts and documentation (<5)

4	Documentation	10										
5	Viva	10										
TOTAL												
Signature of the Faculty member												

EX. No:1	Execution of matrix operations using NumPy to perform in Data analysis.
DATE:	

AIM:

To execute various **matrix operations using NumPy** in Python for performing efficient computations and analysis of data in data analysis applications.

ALGORITHM:

- **Step 1:** Start.
- **Step 2:** Import the NumPy package to enable numerical and matrix operations. ➤
- Step 3:** Define matrices using NumPy arrays to represent data for analysis. ➤ **Step**
- 4:** Execute basic matrix operations such as addition, subtraction, multiplication, transpose, and dot product.
- **Step 5:** Compute statistical measures like mean, sum, standard deviation, and normalize the matrix data.
- **Step 6:** Display the calculated result ➤
- Step 7:** End.

PROGRAM:1 Matrix Operations using NumPy for Data Analysis import

numpy as np

1. Create matrices (arrays)

```
A = np.array([[1, 2, 3],  
             [4, 5, 6],  
             [7, 8, 9]])
```

```
B = np.array([[9, 8, 7],  
             [6, 5, 4],  
             [3, 2, 1]])
```

```
print("Matrix A:\n", A)
```

```
print("\nMatrix B:\n", B) #
```

2. Matrix addition and

subtraction add = A + B

sub = A - B

```
print("\nAddition (A + B):\n", add) print("\nSubtraction  
(A - B):\n", sub)
```

3. Element-wise multiplication and division

mul = A * B div = A / B

```
print("\nElement-wise Multiplication:\n", mul) print("\nElement-wise  
Division:\n", div)
```

4. Matrix multiplication (Dot product) dot_product

= np.dot(A, B)

```
print("\nMatrix Multiplication (Dot Product):\n", dot_product)
```

5. Transpose of a matrix transpose_A

= A.T

```
print("\nTranspose of A:\n", transpose_A)
```

6. Statistical operations (important in data

analysis) print("\nMean of A:", np.mean(A))

```
print("Median of A:", np.median(A)) print("Standard
```

```
Deviation of A:", np.std(A)) print("Variance of A:",
```

```
np.var(A))
```



```
# Row-wise and Column-wise mean print("\nRow-  
wise Mean of A:", np.mean(A, axis=1))  
print("Column-wise Mean of A:", np.mean(A, axis=0))
```

```
# 7. Maximum and Minimum values print("\nMaximum  
value in A:", np.max(A)) print("Minimum value in A:",  
np.min(A))
```

```
# 8. Normalization (Feature Scaling) normalized_A  
= (A - np.min(A)) / (np.max(A) - np.min(A))  
print("\nNormalized Matrix A:\n", normalized_A)
```

```
# 9. Determinant and Inverse (Linear Algebra)  
det_A = np.linalg.det(A) print("\nDeterminant  
of A:", det_A)
```

```
# Inverse exists only if determinant is not zero  
if det_A != 0:  
    inverse_A = np.linalg.inv(A) print("\nInverse of  
A:\n", inverse_A) else: print("\nMatrix A is singular,  
inverse does not exist.")
```

```
# 10. Eigenvalues and Eigenvectors (used in  
PCA) eigenvalues, eigenvector = np.linalg.eig(A)  
print("\nEigenvalues of A:\n", eigenvalues)  
print("\nEigenvectors of A:\n", eigenvector)
```

Output:

```
In [21]: %runfile D:/SKCET/ASP/untitled7.py --wdir
```

Matrix A:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Matrix B:

```
[[9 8 7]
 [6 5 4]
 [3 2 1]]
```

Addition (A + B):

```
[[10 10 10]
 [10 10 10]
 [10 10 10]]
```

Subtraction (A - B):

```
[[ -8 -6 -4]
 [ -2  0  2]
 [  4  6  8]]
```

Element-wise Multiplication:

```
[[ 9 16 21]
 [24 25 24]
 [21 16  9]]
```

Element-wise Division:

```
[[0.11111111 0.25      0.42857143]
 [0.66666667 1.        1.5       ]
 [2.33333333 4.        9.        ]]
```

Matrix Multiplication (Dot Product):

```
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]
```

Transpose of A:

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

```
Mean of A: 5.0
Median of A: 5.0
Standard Deviation of A: 2.581988897471611
Variance of A: 6.666666666666667
```

```
Row-wise Mean of A: [2. 5. 8.]
Column-wise Mean of A: [4. 5. 6.]
```

```
Maximum value in A: 9
Minimum value in A: 1
```

```
Normalized Matrix A:
[[0.    0.125 0.25 ]
 [0.375 0.5   0.625]
 [0.75  0.875 1.    ]]
```

```
Determinant of A: 6.66133814775094e-16
```

```
Inverse of A:
[[-4.50359963e+15  9.00719925e+15 -4.50359963e+15]
 [ 9.00719925e+15 -1.80143985e+16  9.00719925e+15]
 [-4.50359963e+15  9.00719925e+15 -4.50359963e+15]]
```

```
Eigenvalues of A:
[ 1.61168440e+01 -1.11684397e+00 -3.47820491e-16]
```

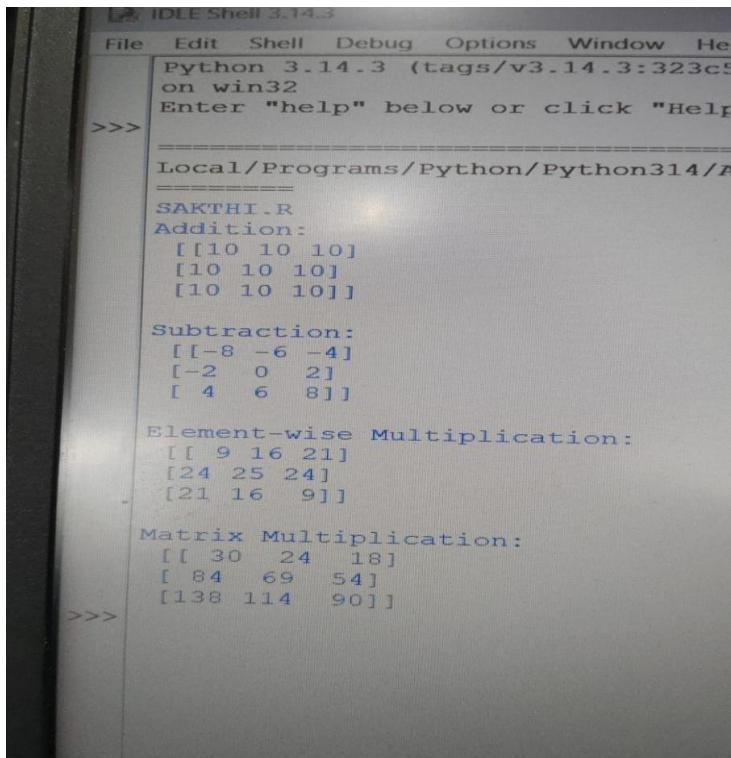
```
Eigenvectors of A:
[[-0.23197069 -0.78583024  0.40824829]
 [-0.52532209 -0.08675134 -0.81649658]
 [-0.8186735   0.61232756  0.40824829]]
```

Result:

The program successfully performs matrix operations such as addition, multiplication, transpose, and statistical calculations using NumPy.

EXERCISE:

1. Create two 3×3 matrices using NumPy and perform addition, subtraction, element-wise multiplication, and matrix multiplication.



```
Python 3.14.3 (tags/v3.14.3:323c5)
on win32
Enter "help" below or click "Help"

>>>

Local/Programs/Python/Python314/

SAKTHI.R
Addition:
[[10 10 10]
 [10 10 10]
 [10 10 10]]

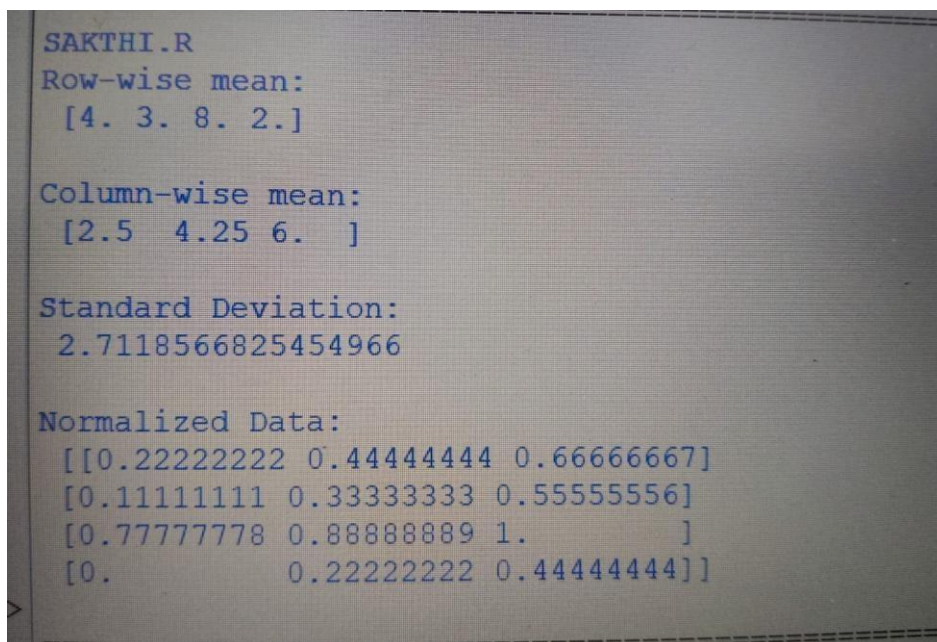
Subtraction:
[[-8 -6 -4]
 [-2  0  2]
 [ 4  6  8]]

Element-wise Multiplication:
[[ 9 16 21]
 [24 25 24]
 [21 16  9]]

Matrix Multiplication:
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]

>>>
```

2. Given a 4×3 dataset matrix, calculate the row-wise mean, column-wise mean, standard deviation, and normalize the data.



```
SAKTHI.R
Row-wise mean:
[4.  3.  8.  2.]

Column-wise mean:
[2.5  4.25  6.  ]

Standard Deviation:
2.7118566825454966

Normalized Data:
[[0.22222222 0.44444444 0.66666667]
 [0.11111111 0.33333333 0.55555556]
 [0.77777778 0.88888889 1.         ]
 [0.         0.22222222 0.44444444]]

>
```

3. Write a NumPy program to find the determinant and inverse of a 2×2 matrix and solve a system of linear equations using `np.linalg.solve()`.

```
SAKTHI.R

Determinant of A: 5.000000000000001

Inverse of A:
[[ 0.8 -0.6]
 [-0.2  0.4]]

Linear Equation:
[2.2 1.2]
```

Viva questions:

1. What is NumPy and why is it used in data analysis?
2. What is the difference between a NumPy array and a Python list?
3. How do you create a matrix in NumPy?
4. What is broadcasting in NumPy?
5. How is matrix multiplication performed in NumPy?
6. What is the difference between dot() and matmul()?
7. How do you find the transpose of a matrix?
8. What are eigenvalues and eigenvectors?
9. How do you compute the inverse of a matrix?
10. What errors can occur during matrix operations?

EX.No:2	Computation of mean, variance and standard deviation using Python.
DATE:	

AIM:

To write a Python program to compute mean, variance and standard deviation.

ALGORITHM:

- Step 1: Start
- Step 2: import the NumPy library.
- Step 3: Read the number of elements from the user.
- Step 4: Accept the data values from the user and store them in an array. ➤ Step 5: Compute the mean, variance, and standard deviation using NumPy functions.
- Step 6: Display the calculated statistical values. ➤

Step 6: End

PROGRAM:

```
import numpy as np
# Get number of elements from user n =
int(input("Enter number of elements: "))

# Get data values from
user data = [] for i in
range(n):
    value = float(input(f"Enter element {i+1}: "))
data.append(value)
# Convert list to NumPy array
data = np.array(data)

# Compute statistics
mean = np.mean(data)
variance = np.var(data)
std_deviation = np.std(data)
```

```
# Display results print("\nData:", data)
print("Mean:", mean) print("Variance:",
variance) print("Standard Deviation:",
std_deviation)
```

Output:

```
In [22]: %runfile D:/SKCET/ASP/untitled8.py --wdir
Enter number of elements: 5
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
Enter element 5: 50

Data: [10. 20. 30. 40. 50.]
Mean: 30.0
Variance: 200.0
Standard Deviation: 14.142135623730951
```

Result:

The program successfully calculates the mean, variance, and standard deviation for the user entered data. It displays accurate statistical results that help analyze the distribution and spread of the data values.

EXERCISE:

1. Write a Python program to calculate the mean and standard deviation of a list of student marks entered by the user.

```
===== RESTART: C:/Users/sakth/OneDrive/Desktop/
Enter number of elements:5
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
Enter element 5: 50

SAKTHI.R

Data: [10. 20. 30. 40. 50.]

Mean: 30.0

Variance: 200.0

Standard Deviation: 14.142135623730951
```

2. Modify the program to compute variance and standard deviation for a 2D dataset.

```
===== RESTART: C:/Users/sakth/OneDrive/Desktop/
```

```
SAKTHI.R
```

```
Dataset:
```

```
[[10 20 30]
```

```
[40 50 60]
```

```
[70 80 90]]
```

```
Variance: 666.6666666666666
```

```
Standard Deviation: 25.81988897471611
```

3. Create a program to compare the mean and standard deviation of two different datasets using NumPy.

```
===== RESTART: C:/Users/sakth/OneDrive/Desktop/La
```

```
SAKTHI.R
```

```
Dataset 1: [10 20 30 40 50]
```

```
Mean of Dataset 1: 30.0
```

```
Standard Deviation of Dataset 1: 14.142135623730951
```

```
Dataset 2: [15 25 35 45 55]
```

```
Mean of Dataset 2: 35.0
```

```
Standard Deviation of Dataset 2: 14.142135623730951
```

```
Dataset 2 has higher mean
```

Viva Questions:

1. What is mean and how is it calculated?
2. What does variance indicate about a dataset?
3. Why is standard deviation more interpretable than variance?
4. How do population and sample variance differ?
5. Which NumPy functions compute mean and standard deviation?
6. What happens to mean if outliers are present?
7. Can standard deviation be negative? Why?
8. What is the relationship between variance and standard deviation?
9. How is variance affected by data scaling?
10. Where are these measures used in real-world applications?

EX.No:3	Data visualization for univariate and bivariate datasets.
DATE:	

AIM:

To write a python program for data visualization for univariate and bivariate datasets.

ALGORITHM:

- **Step 1:**Start the program
- **Step 2:**import required libraries such as NumPy and Matplotlib.
- **Step 3:**Create or read the univariate and bivariate datasets.
- **Step 4:**Plot a histogram to visualize the univariate data distribution. ➤ **Step 5:** Plot scatter and line graphs to represent the relationship between bivariate data.
- **Step 6:** Display all the graphs with appropriate titles and axis labels.
- **Step 7:** End.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
# Sample univariate dataset
data = np.array([10, 15, 20, 25, 30, 35, 40])

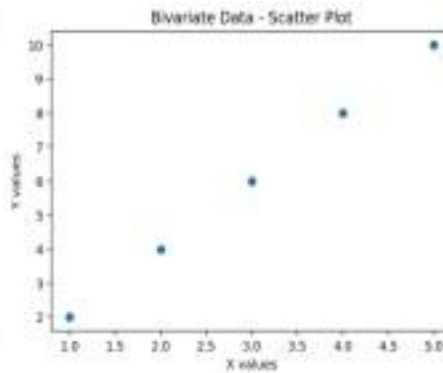
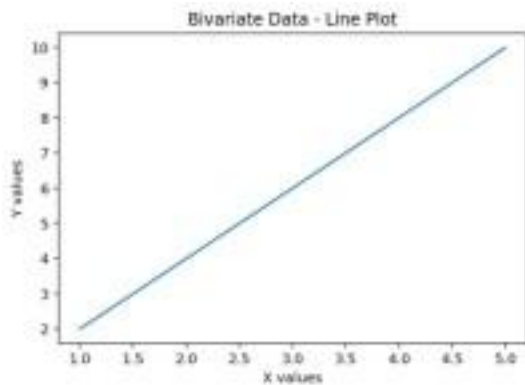
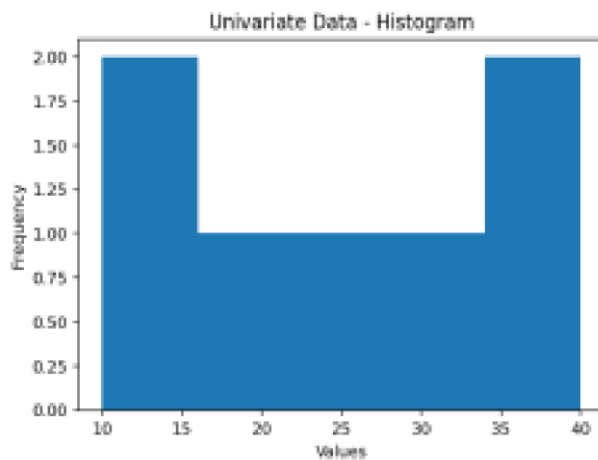
# Histogram
plt.figure()
plt.hist(data, bins=5)
plt.title("Univariate Data - Histogram")
plt.xlabel("Values")
plt.ylabel("Frequency")
```

```
plt.show() # Sample
bivariate dataset x =
np.array([1, 2, 3, 4, 5]) y =
np.array([2, 4, 6, 8, 10])
```

```
# Scatter Plot plt.figure()
plt.scatter(x, y) plt.title("Bivariate
Data - Scatter Plot") plt.xlabel("X
values") plt.ylabel("Y values")
plt.show()
```

```
# Line Plot plt.figure() plt.plot(x,
y) plt.title("Bivariate Data - Line
Plot") plt.xlabel("X values")
plt.ylabel("Y values") plt.show()
```

Output:

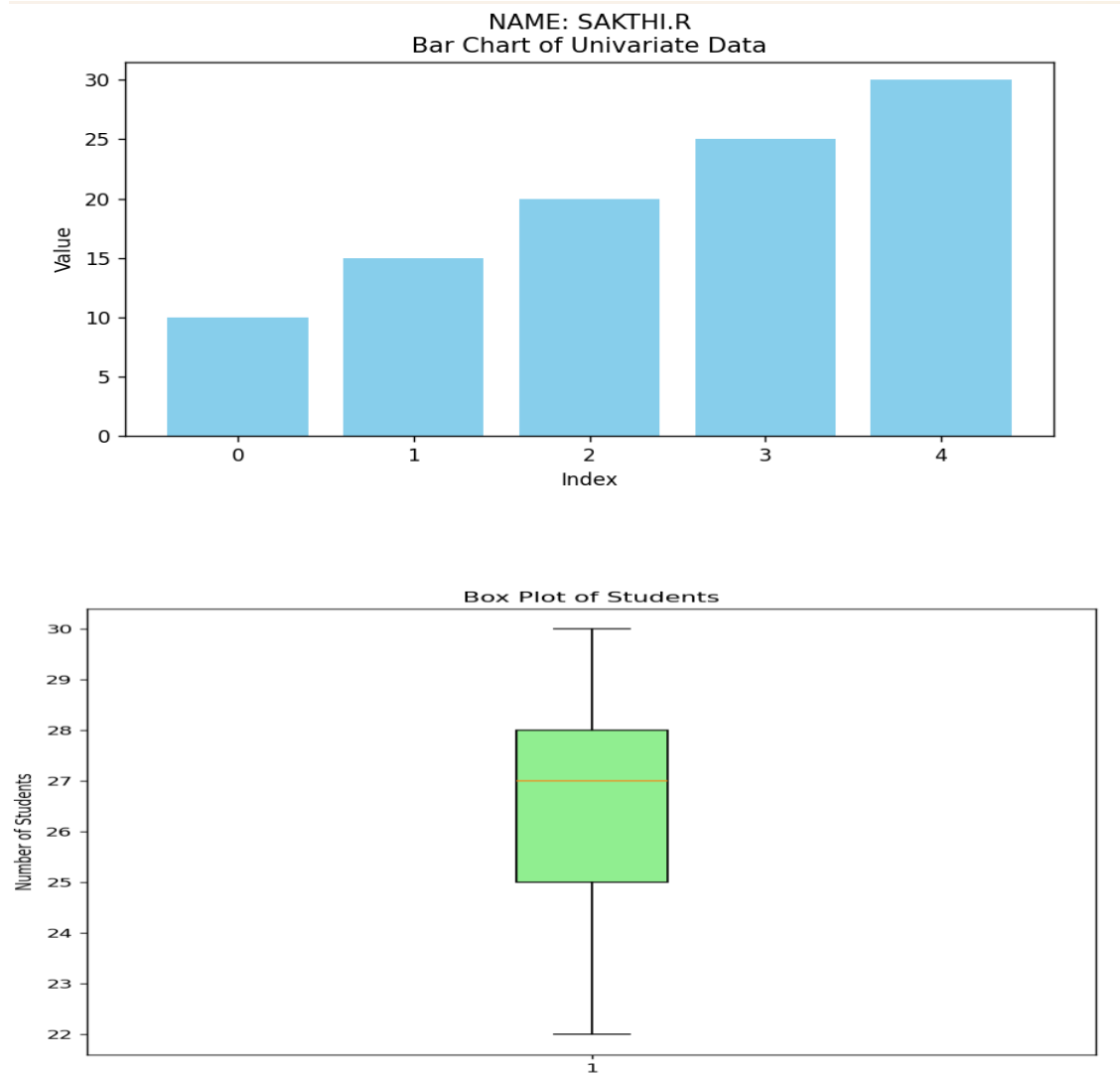


RESULT:

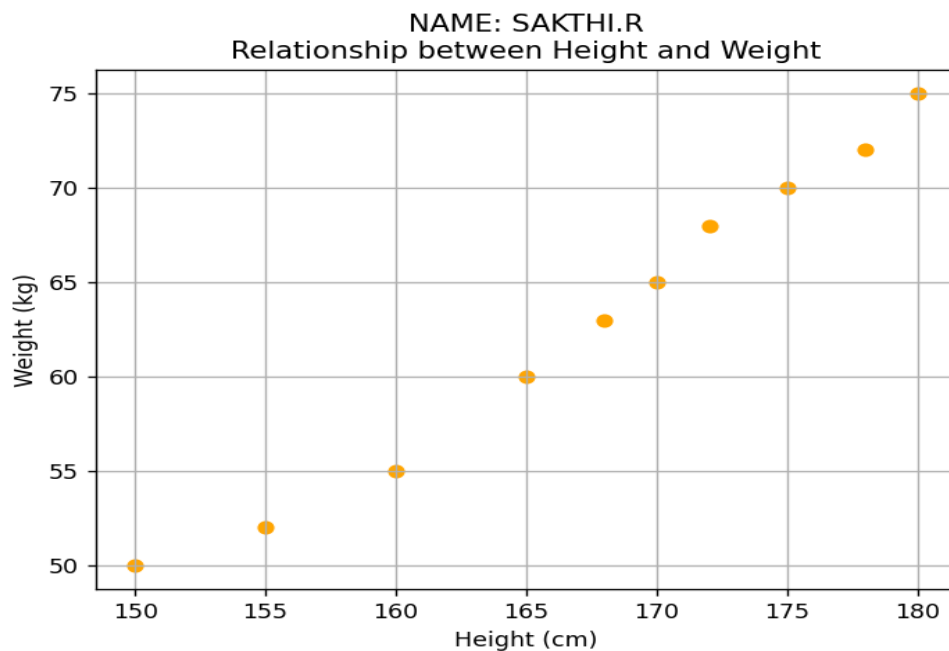
The program successfully visualizes univariate data using a histogram and bivariate data using scatter and line plots. These visualizations clearly represent data distribution and relationships, aiding effective data analysis.

Exercise:

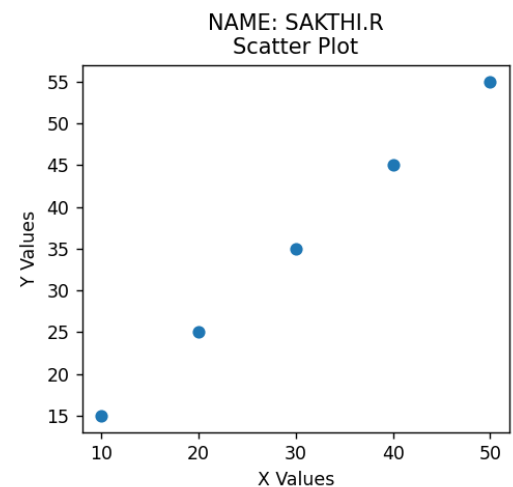
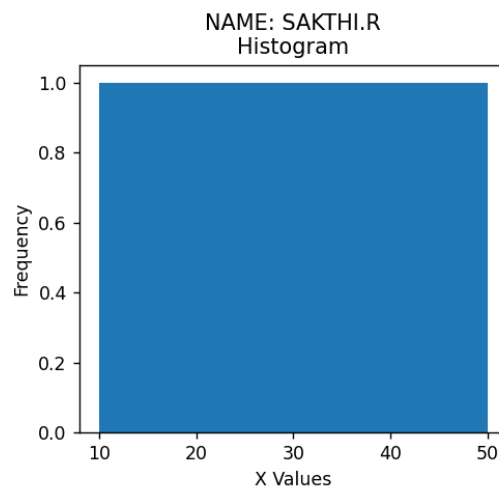
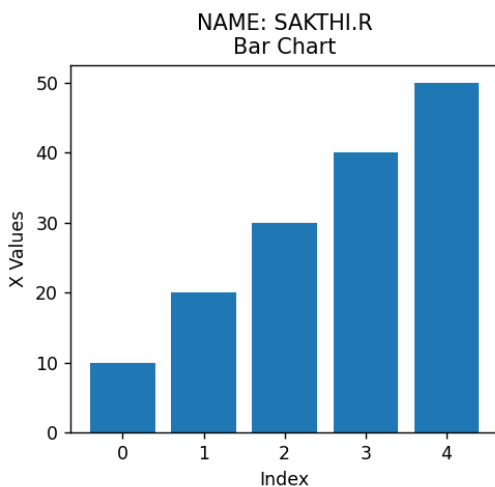
1. Write a Python program to visualize univariate data using a bar chart and box plot.



2. Create a bivariate visualization program using a scatter plot to show the relationship between height and weight.



3. Modify the program to accept user input data and visualize it using appropriate univariate and bivariate plots.



Viva Questions:

1. What is data visualization?
2. What is univariate data?
3. What is bivariate data?
4. Which plots are used for univariate analysis?
5. Which plots are used for bivariate analysis?
6. What is a histogram?
7. What is a scatter plot?
8. Why is data visualization important?
9. What libraries are used for visualization in Python?
10. How do visualizations help in decision-making?

EX.No:4	Correlation and regression analysis using python.
DATE:	

AIM:

To write a program to calculate correlation coefficient, generates a regression equation and graph to visualize and predict data trends.

ALGORITHM:

- **Step 1:** Start the program
- **Step 2:** import required libraries such as NumPy and Matplotlib.
- **Step 3:** Read or define the two related datasets (X and Y values).
- **Step 4:** Compute the correlation coefficient using NumPy to measure the relationship.
- **Step 5:** Perform linear regression to find the best-fit line and regression equation. ➤
- Step 5:** Display the correlation result and plot the regression line with data points. ➤
- Step 5:** End.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
x = np.array([10, 20, 30, 40, 50])
y = np.array([15, 25, 35, 45, 55])

correlation = np.corrcoef(x, y)
print("Correlation Matrix:\n", correlation)
```

```

# Calculate regression coefficients (y = mx + c) m,
c = np.polyfit(x, y, 1)

print("\nRegression Equation: y = {:.2f}x + {:.2f}".format(m, c)) #
Predicted values
y_pred = m * x + c

# Plot regression line

plt.figure() plt.scatter(x, y)

plt.plot(x, y_pred)

plt.title("Regression Analysis")

plt.xlabel("X values")

plt.ylabel("Y values")

plt.show()

```

OUTPUT:

```

In [8]: %runfile D:/SKCET/ASP/untitled2.py --wdir
Correlation Matrix:
[[1. 1.]
 [1. 1.]]

Regression Equation: y = 1.00x + 5.00

```

RESULT:

The program calculates the correlation coefficient showing the strength of the relationship between variables. It also generates a regression equation and graph to visualize and predict data trends.

Exercise:

1. Write a Python program to find the correlation coefficient between students' study hours and marks obtained.

```
===== RESTART: C:/
Name: SAKTHI.R

Study Hours: [2, 4, 6, 8, 10]
Marks: [20, 40, 60, 80, 100]
Correlation Coefficient: 1.0
```

2. Modify the regression program to accept user input values for X and Y datasets.

```
===== RESTART: C:/Users/sakth/AppData/Local/Programs/Python/Python38-32/Python.exe
Name: SAKTHI.R
Enter X values (space separated): 2 4 6 8 10
Enter Y values (space separated): 20 40 60 80 100

Regression Equation: Y = 0.0 + 10.0 * X
```

3. Create a program to compare regression lines for two different datasets and analyze the results.

```
===== RESTART: C:/Users/sakth/AppData/Local/Programs/Python/Python38-32/Python.exe
Name: SAKTHI.R
Dataset 1 Regression Line: Y = 2.2 + 0.6 * X
Dataset 2 Regression Line: Y = 4.0 + 1.0 * X

Analysis:
Dataset 2 has a steeper slope than Dataset 1.
The intercepts are different.
>> |
```

Viva Questions:

1. What is correlation?
2. What are the types of correlation?
3. What is Pearson correlation coefficient?
4. What is regression analysis?
5. Difference between correlation and regression?
6. What is linear regression?
7. What does slope represent in regression?
8. What is R-squared value?
9. Can correlation exist without causation?
10. What are real-life applications of regression?

EX.NO:5	Hypothesis testing using t-test and chi-square test
DATE:	

AIM:

To conduct hypothesis testing using:

- T-Test (for small and large samples)
- Chi-Square Test (for categorical data)

ALGORITHM:

T-Test:

- Step 1: Input sample data for two groups.
- Step 2: Calculate means and standard deviations.
- Step 3: Apply t-test using `scipy.stats.ttest_ind()`.
- Step 4: Print t-statistic and p-value.
- Step 5: Interpret the p-value to accept/reject null hypothesis.

Chi-Square Test:

- Step 1: Input observed data in matrix form.
- Step 2: Use `scipy.stats.chi2_contingency()`.
- Step 3: Calculate chi-square statistic, p-value, degrees of freedom.
- Step 4: Interpret results using significance level.

PROGRAM:

· _____ T-

Test Using Python: from

`scipy import stats`

```
n = int(input("Enter number of elements in each group: "))
group1 = [float(input(f"Group1 value {i+1}: ")) for i in range(n)]
group2 = [float(input(f"Group2 value {i+1}: ")) for i in range(n)]
t_stat, p_value = stats.ttest_ind(group1, group2)
```

```
print("T-statistic:", t_stat)
print("P-value:", p_value)
if p_value < 0.05:
    print("Reject H0: Significant difference.")
else:
    print("Fail to reject H0: No significant difference.")
```

· _____ Chi-Square Test Using Python:


```

import numpy as np
from scipy.stats import chi2_contingency

rows = int(input("Enter number of rows: ")) cols
= int(input("Enter number of columns: "))
observed = []

for i in range(rows): row = [int(input(f"Value at ({i+1},{j+1}):
")) for j in range(cols)] observed.append(row)

observed_array = np.array(observed)
chi2, p, dof, expected = chi2_contingency(observed_array)

print("Chi-Square Value:", chi2)
print("P-value:", p) print("Degrees of
Freedom:", dof) print("Expected
Frequencies:\n", expected)

if p < 0.05:
    print("Reject H0: Variables are dependent.")
else:
    print("Fail to reject H0: Variables are independent.")

```

OUTPUT:

t-test:

```

In [23]: %runfile D:/SKCET/ASP/untitled9.py --wdir
Enter number of elements in each group: 5
Group1 value 1: 20
Group1 value 2: 30
Group1 value 3: 40
Group1 value 4: 60
Group1 value 5: 20
Group2 value 1: 10
Group2 value 2: 40
Group2 value 3: 60
Group2 value 4: 20
Group2 value 5: 80
T-statistic: -0.5393598899705937
P-value: 0.6043209674870225
Fail to reject H0: No significant difference.

```

Chi-test:

```

In [24]: %runfile D:/SKCET/ASP/untitled10.py --wdir
Enter number of rows: 3
Enter number of columns: 3
Value at (1,1): 2
Value at (1,2): 4
Value at (1,3): 6
Value at (2,1): 3
Value at (2,2): 5
Value at (2,3): 7
Value at (3,1): 8
Value at (3,2): 9
Value at (3,3): 2
Chi-Square Value: 7.646387764282502
P-value: 0.1054247825060586
Degrees of Freedom: 4
Expected Frequencies:
[[3.39130435 4.69565217 3.91304348]
 [4.23913043 5.86956522 4.89130435]
 [5.36956522 7.43478261 6.19565217]]
Fail to reject H0: Variables are independent.

```

RESULT:

The hypothesis testing was successfully conducted using Python for:

- T-Test on small and large samples
- Chi-Square Test on categorical data

EXERCISE:

1.Modify the t-test program to handle unequal sample sizes.

```

=====
cal/Programs/Python/Python314/ASUP Exp 5.py ===
=====
Name: SAKTHI.R
Sample 1: [12, 15, 14, 10, 13]
Sample 2: [22, 25, 21, 24]
t-value: -8.1319
Degrees of freedom: 6.72
> |

```

2.Implement a one-sample t-test to check if the sample mean is significantly different from a given value.

```

=====
cal/Programs/Python/Python314/ASUP Exp 5.py =====
=====
Name: SAKTHI.R
Sample: [10, 12, 9, 11, 10, 13, 12]
Hypothesized Mean: 10
Sample Mean: 11.0
t-value: 1.8708
Degrees of freedom: 6
>

```

3. Extend the chi-square program to handle goodness-of-fit test.

```

===== RESTART: C:/Users/sakth/AppData/Local/P:
Name: SAKTHI.R
Observed: [20, 30, 25, 25]
Expected: [25, 25, 25, 25]
Chi-square value: 2.0
Degrees of freedom: 3
>

```

4. Use your chi-square program to analyze real-world data such as survey responses.

```

===== RESTART: C:/Users/sakth/
Name: SAKTHI
Observed counts: [30, 25, 20, 25]
Expected counts: [25, 25, 25, 25]
Chi-square value: 2.0
Degrees of freedom: 3
Result: Fail to reject H0 – preferences are not significantly different.
|

```

Viva Questions:

1. What is hypothesis testing?
2. Define null hypothesis and alternative hypothesis.
3. What is a t-test?
4. What are the types of t-tests?
5. What is Chi-square test?
6. When is Chi-square test used?
7. What is significance level?
8. What is a p-value?
9. What happens if $p\text{-value} < \alpha$?
10. What assumptions are required for t-test?

EX.NO:6	Analysis of variance - based comparison of datasets.
DATE:	

AIM:

To perform Analysis of Variance (ANOVA) using Python in order to compare multiple datasets and determine whether there is a statistically significant difference among their means.

ALGORITHM:

- Step 1: Start the program and import required libraries such as NumPy and SciPy.
- Step 2: Define or input multiple datasets representing different groups.
- Step 3: Apply the one-way ANOVA test using stats.f_oneway() function.
- Step 4: Obtain the F-statistic and p-value from the ANOVA test.
- Step 5: Compare the p-value with the significance level (0.05) and display the result.
- Step 6: End the program

PROGRAM:

```
import numpy as np
from scipy import stats

# Sample datasets (e.g., scores from three different groups)
group1 = np.array([85, 88, 90, 92, 86])
group2 = np.array([78, 80, 82, 79, 81])
group3 = np.array([90, 92, 94, 91, 93])

# Perform One-Way ANOVA
f_statistic, p_value = stats.f_oneway(group1, group2, group3)

# Display results
print("Group 1:", group1)
print("Group 2:", group2)
print("Group 3:", group3)

print("\nF-Statistic:", f_statistic)
print("P-Value:", p_value)
```

```
# Interpretation
```

```
if p_value < 0.05:
```

```
print("\nResult: Significant difference exists between the group means.")
```

```
else: print("\nResult: No significant difference exists between the group means.")
```

OUTPUT:

```
In [10]: %runfile D:/SKCET/ASP/untitled3.py --wdir
Group 1: [85 88 90 92 86]
Group 2: [78 80 82 79 81]
Group 3: [90 92 94 91 93]

F-Statistic: 42.74242424242423
P-Value: 3.479082682504061e-06

Result: Significant difference exists between the group means.
```

RESULT:

The program successfully computes the F-statistic and p-value for the given datasets. Based on the p-value, it determines whether a significant difference exists between the group means.

EXERCISE:

1. Write a Python program to perform ANOVA on the marks of students from four different classes.

```
===== RESTART: C:
Name: SAKTHI.R
Group Means: [88.4, 76.8, 92.8, 68.8]
Overall Mean: 81.7
SSB (Between Groups): 1792.6
SSW (Within Groups): 93.6
Degrees of Freedom Between: 3
Degrees of Freedom Within: 16
MSB (Mean Square Between): 597.53
MSW (Mean Square Within): 5.85
F-value: 102.1425
|
```

2. Modify the program to accept datasets as user input and perform one-way ANOVA.

```

----- RESTART: C:/Users/sakth/AppData/Local/Programs/Python/Python314/ASUP Exp 5.py =====
=====
Name: SAKTHI.R
Enter the number of classes: 4
Enter marks for class 1 separated by spaces: 85 90 88 92 87
Enter marks for class 2 separated by spaces: 78 75 80 74 77
Enter marks for class 3 separated by spaces: 92 95 94 90 93
Enter marks for class 4 separated by spaces: 70 72 68 65 69

--- ANOVA Results ---
Mean of Class 1: 88.4
Mean of Class 2: 76.8
Mean of Class 3: 92.8
Mean of Class 4: 68.8
Overall Mean: 81.7
SSB (Between Groups): 1792.6
SSW (Within Groups): 93.6
Degrees of Freedom Between: 3
Degrees of Freedom Within: 16
MSB (Mean Square Between): 597.53
MSW (Mean Square Within): 5.85
F-value: 102.1425
Interpretation: Large F-value indicates significant difference between class means.

```

3. Create a program to perform two-way ANOVA and analyze the interaction between two independent factors.

```

===== RESTART: C:/Users/sakth/AppData/Local/Programs/Python
Name: SAKTHI.R
Factor A (Method) SS=288.00, df=1, MS=288.00, F=26.79
Factor B (Class Level) SS=4.50, df=1, MS=4.50, F=0.42
Interaction SS=2.00, df=1, MS=2.00, F=0.19
Within/Error SS=43.00, df=4, MS=10.75

```

Viva Questions:

1. What is ANOVA?
2. Why is ANOVA used?
3. What is the null hypothesis in ANOVA?
4. What is F-statistic?
5. Difference between ANOVA and t-test?
6. What are types of ANOVA?
7. What assumptions does ANOVA make?
8. What is within-group variance?
9. What is between-group variance?
10. Where is ANOVA applied in engineering?

EX.NO:7	Principal Component Analysis using Python.
DATE:	

AIM:

To apply Principal Component Analysis (PCA) using Python to reduce the dimensionality of a dataset while retaining maximum variance for effective data analysis.

ALGORITHM:

1. Start the program and import required libraries such as NumPy and Scikit-learn.
2. Create or input the dataset containing multiple features.
3. Standardize the dataset to bring all features to the same scale.
4. Apply PCA to transform the data into principal components.
5. Display the principal components and explained variance ratio.

PROGRAM:

```
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
# Sample dataset (rows = observations, columns = features) data
```

```
= np.array([
```

```
    [2.5, 2.4],
```

```
    [0.5, 0.7],
```

```
    [2.2, 2.9],
```

```
    [1.9, 2.2],
```

```
    [3.1, 3.0],
```

```
    [2.3, 2.7],
```

```
    [2.0, 1.6],
```

```
    [1.0, 1.1],
```

```
    [1.5, 1.6],
```

```
    [1.1, 0.9]
```

```
])
```

```
# Step 1: Standardize the data scaler =
```

```
StandardScaler() data_scaled =
```

```
scaler.fit_transform(data)
```

```
# Step 2: Apply PCA
pca = PCA(n_components=2)
principal_components = pca.fit_transform(data_scaled)

# Display results
print("Original Data:\n", data)
print("\nStandardized Data:\n", data_scaled)
print("\nPrincipal Components:\n",
principal_components)

# Explained variance
print("\nExplained Variance Ratio:", pca.explained_variance_ratio_)
```

OUTPUT:

```
In [11]: %runfile D:/SKCET/ASP/untitled4.py --wdir
Original Data:
[[2.5 2.4]
 [0.5 0.7]
 [2.2 2.9]
 [1.9 2.2]
 [3.1 3. ]
 [2.3 2.7]
 [2.  1.6]
 [1.  1.1]
 [1.5 1.6]
 [1.1 0.9]]

Standardized Data:
[[ 0.92627881  0.61016865]
 [-1.7585873  -1.506743  ]
 [ 0.52354889  1.23278973]
 [ 0.12081898  0.36112022]
 [ 1.73173864  1.35731394]
 [ 0.6577922   0.9837413  ]
 [ 0.25506228 -0.38602507]
```

```
[ 1.73173864  1.35731394]
 [ 0.6577922   0.9837413  ]
 [ 0.25506228 -0.38602507]
 [-1.08737078 -1.00864614]
 [-0.41615425 -0.38602507]
 [-0.95312747 -1.25769457]]
```

```
Principal Components:
[[ 1.08643242  0.22352364]
 [-2.3089372  -0.17808082]
 [ 1.24191895 -0.501509  ]
 [ 0.34078247 -0.16991864]
 [ 2.18429003  0.26475825]
 [ 1.16073946 -0.23048082]
 [-0.09260467  0.45331721]
 [-1.48210777 -0.05566672]
 [-0.56722643 -0.02130455]
 [-1.56328726  0.21536146]]
```

```
Explained Variance Ratio: [0.96296464 0.03703536]
```


RESULT:

The program successfully transforms the original dataset into principal components. It also displays the explained variance ratio, indicating the amount of information preserved after dimensionality reduction.

EXERCISE:

1. Write a Python program to apply PCA on a dataset with three or more features.

```
===== RESTART: C:/Us
cal/Programs/Python/Python314/ASUP EXP 7.py =====
=====
Name: SAKTHI.R
Original Dataset:
[[2.5 2.4 1.2 3.5]
 [0.5 0.7 0.3 1.2]
 [2.2 2.9 1.8 3. ]
 [1.9 2.2 1.5 2.8]
 [3.1 3.  2.  3.6]]

Principal Components:
[[ 6.26322369e-01 -7.75797418e-01]
 [-3.64946300e+00  1.18347087e-02]
 [ 9.64859341e-01  5.32637294e-01]
 [-2.31722490e-03  2.23712127e-01]
 [ 2.06059852e+00  7.61328882e-03]]

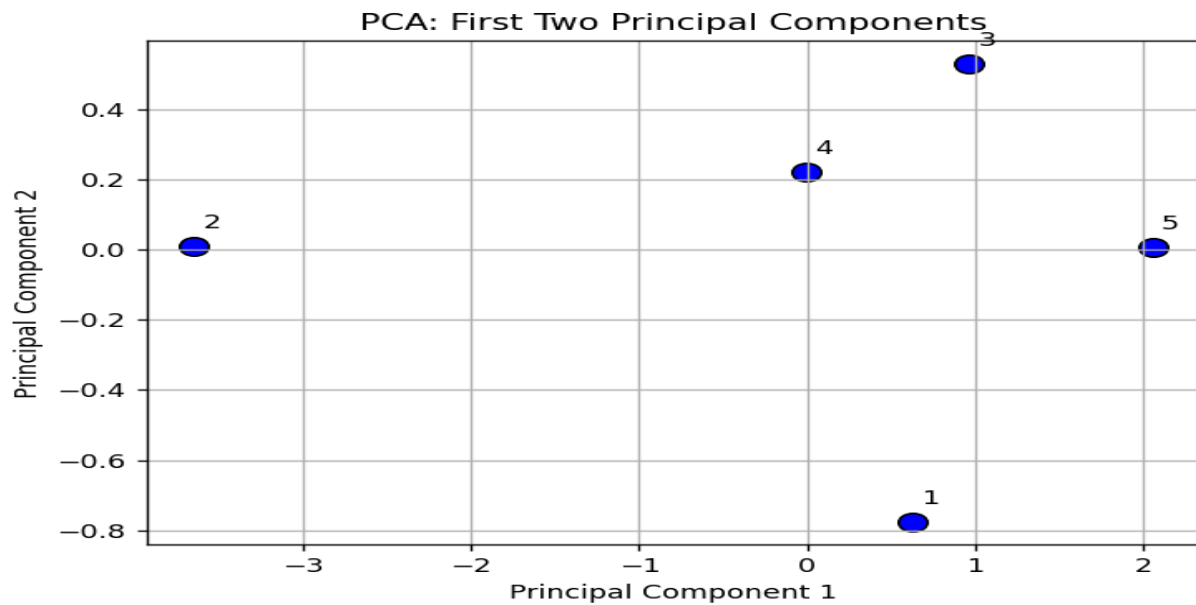
Explained Variance Ratio:
[0.94439425 0.04679046]
```

2. Modify the program to visualize the first two principal components using a scatter plot.

```
===== RESTART: C:/Us
Original Dataset:
[[2.5 2.4 1.2 3.5]
 [0.5 0.7 0.3 1.2]
 [2.2 2.9 1.8 3. ]
 [1.9 2.2 1.5 2.8]
 [3.1 3.  2.  3.6]]

Principal Components:
[[ 6.2630e-01 -7.7580e-01]
 [-3.6495e+00  1.1800e-02]
 [ 9.6490e-01  5.3260e-01]
 [-2.3000e-03  2.2370e-01]
 [ 2.0606e+00  7.6000e-03]]

Explained Variance Ratio:
[0.9444 0.0468]
Name: SAKTHI.R
```



3. Create a program to compare model performance before and after applying PCA.

```
----- RESIDUAL: C:/USERS/SAKTHI
Name: SAKTHI.R
Accuracy on Original Data: 1.0000
Accuracy after PCA: 1.0000
Explained Variance Ratio of PCA: [0.9773 0.0208]
```

Viva Questions:

1. What is PCA?
2. Why is PCA used?
3. What is dimensionality reduction?
4. What are principal components?
5. How are principal components selected?
6. What is explained variance?
7. How does PCA reduce noise?
8. Is PCA supervised or unsupervised?
9. What are the limitations of PCA?
10. Where is PCA used in real life?

EX.NO:8	Application of K-Means clustering for engineering dataset
DATE:	

AIM:

To write a program that performs K-means Clustering

ALGORITHM:

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
```

#Generates synthetic data

```
X,y=make_blobs(n_samples=300,centers=4,cluster_std=0.60,random_state=0)
```

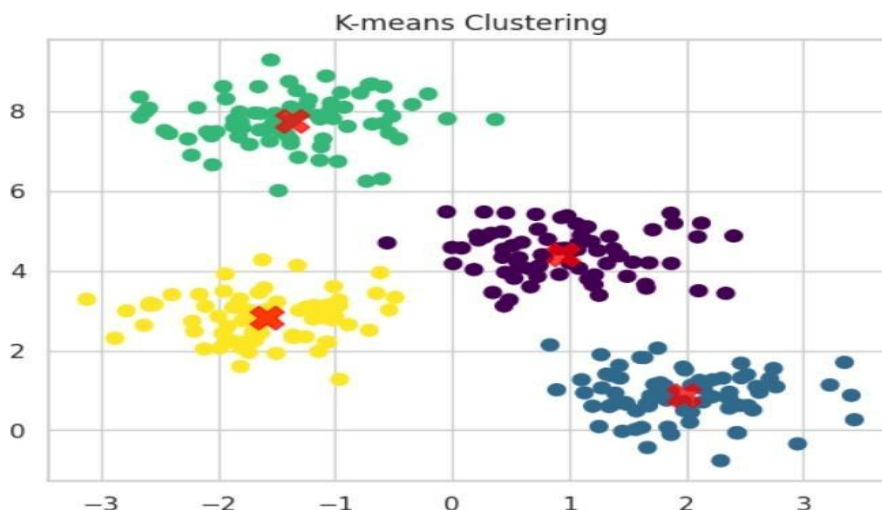
#Fit K-means model

```
kmeans=KMeans(n_clusters=4)
kmeans.fit(X)
y_kmeans=kmeans.predict(X)
```

Plot the results

```
plt.scatter(X[:,0],X[:,1],c=y_kmeans,s=50,cmap='viridis')
centers = kmeans.cluster_centers_
plt.scatter(centers[:,0],centers[:,1],c='red',s=200,alpha=0.75,marker='X')
plt.title('K-means Clustering')
plt.show()
```

OUTPUT



RESULT:

Thus ,python program is successfully implemented that performs K means Clustering using scikit - learn.

Write a program that perform K-means Clustering using scikit-learn

Logistic Regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score
# Load dataset iris = load_iris()
X = iris.data y = iris.target

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and fit the model
model = LogisticRegression(max_iter=200) model.fit(X_train, y_train)

# Predict and evaluate
y_pred = model.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, y_pred)}') print(f'Confusion
Matrix:\n{confusion_matrix(y_test, y_pred)}')
```

```
Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Time Series Forecasting with ARIMA

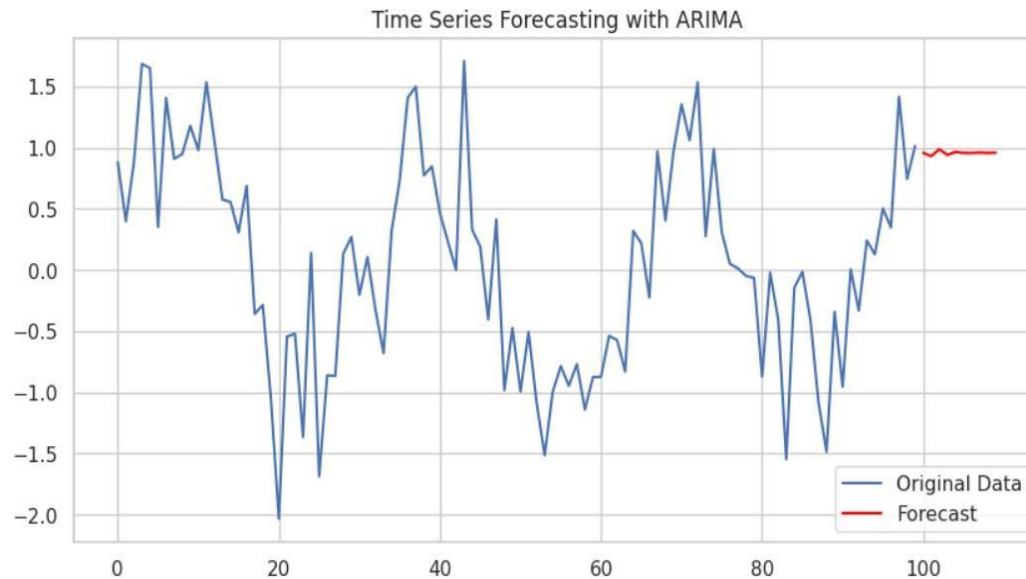
```
import numpy as np import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
# Generate a simple time series data np.random.seed(0)
time = np.arange(100)
data = np.sin(time / 5) + np.random.normal(scale=0.5, size=len(time))

# Create a DataFrame
df = pd.DataFrame(data, columns=['value']) df['time'] = time

# Fit ARIMA model
model = ARIMA(df['value'], order=(5, 1, 0)) model_fit = model.fit()
# Forecast
forecast = model_fit.forecast(steps=10)

# Plot plt.figure(figsize=(10, 5))
plt.plot(df['value'], label='Original Data')
```

```
plt.plot(np.arange(len(df), len(df) + 10), forecast, label='Forecast', color='red')
plt.legend()
plt.title('Time Series Forecasting with ARIMA')
plt.show
```



We create the below loop to test and store different model results so that we can make a decision on the best number of clusters.

```
K = range(2, 8)
fits = []
score = []
```

```
for k in K:
```

```
    # train the model for current value of k on training data
```

```
model = KMeans(n_clusters = k, random_state = 0, n_init='auto').fit(X_train_norm)
```

```
    # append the model to fits
```

```
    fits.append(model)
```

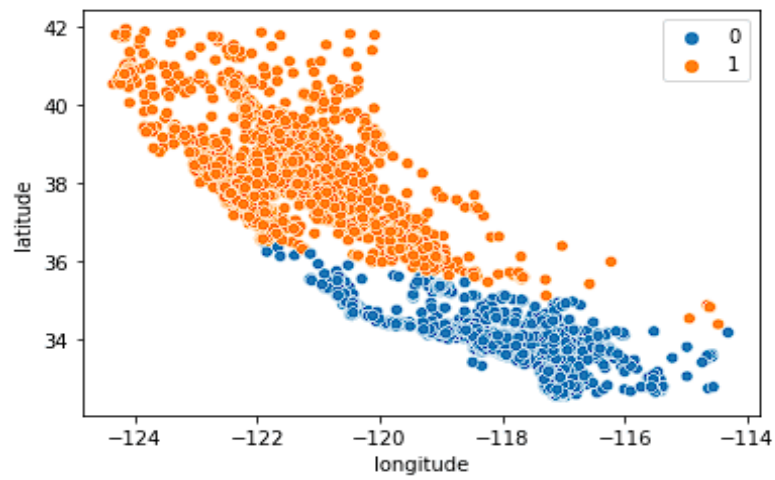
```
    # Append the silhouette score to scores
```

```
score.append(silhouette_score(X_train_norm, model.labels_, metric='euclidean'))
```

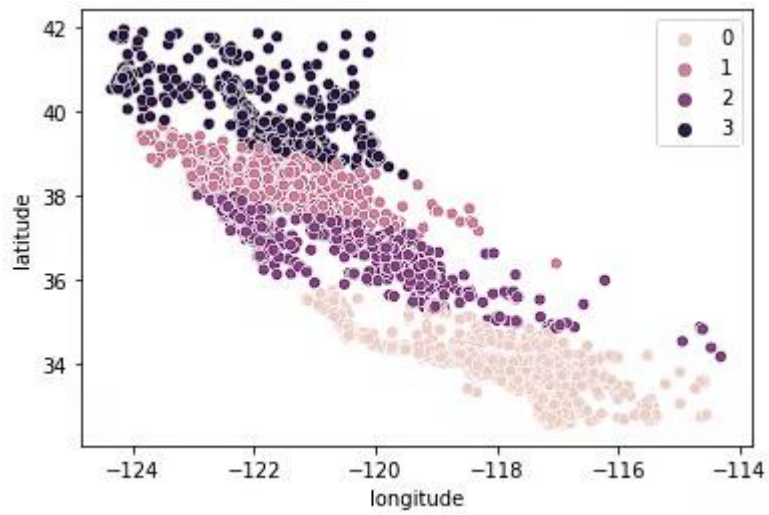
We can then first visually look at a few different values of k.

First we look at k = 2.

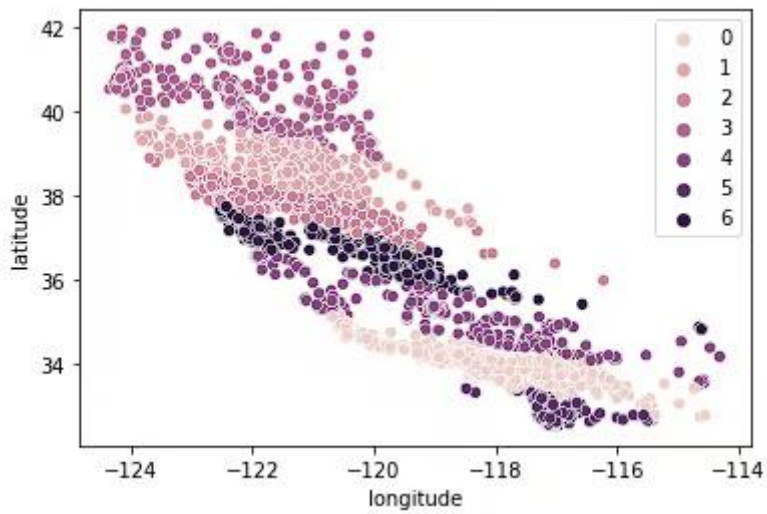
```
sns.scatterplot(data = X_train, x = 'longitude', y = 'latitude', hue=fits[0].labels_)
```



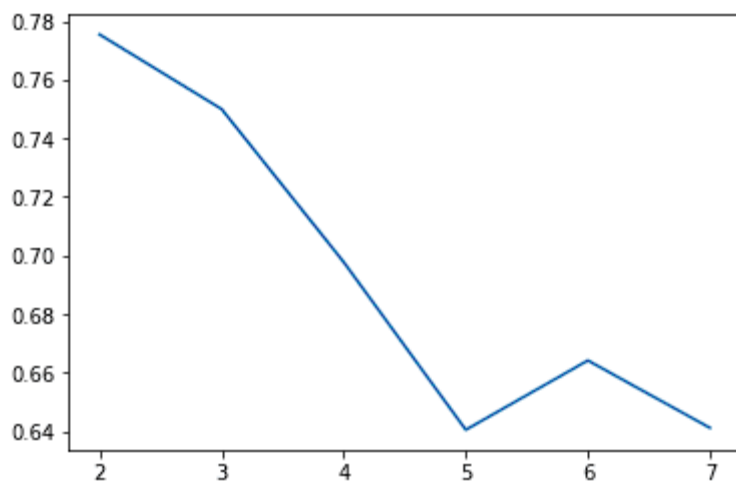
Next, we look at $k = 4$.



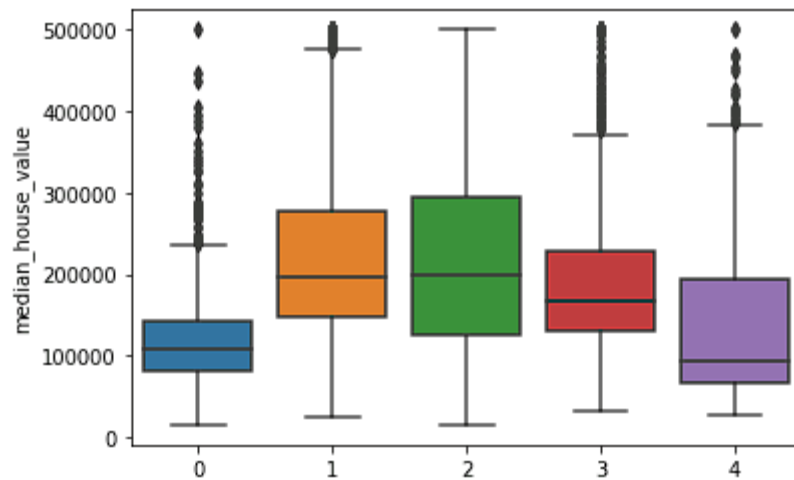
Finally, we look at $k = 7$.



`sns.lineplot(x = K, y = score)`

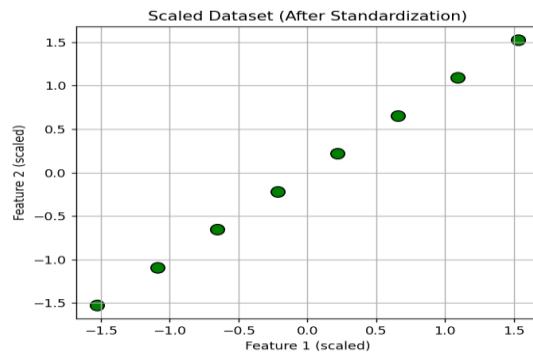
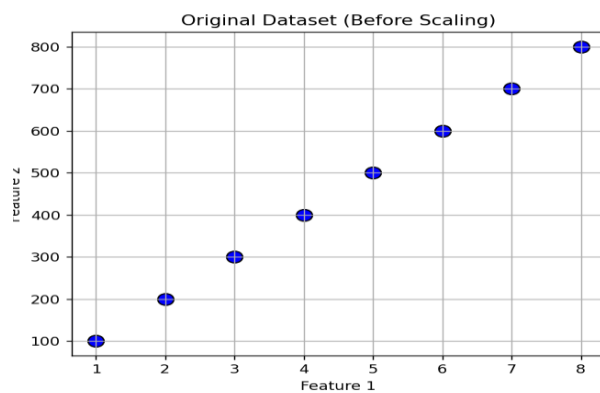


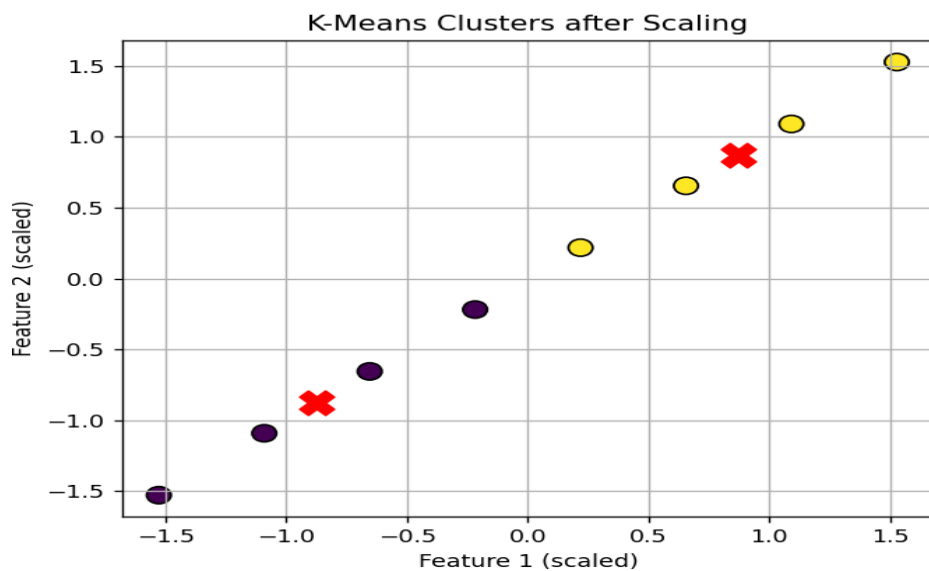
```
sns.boxplot(x = fits[3].labels_, y = y_train['median_house_value'])
```



EXERCISE:

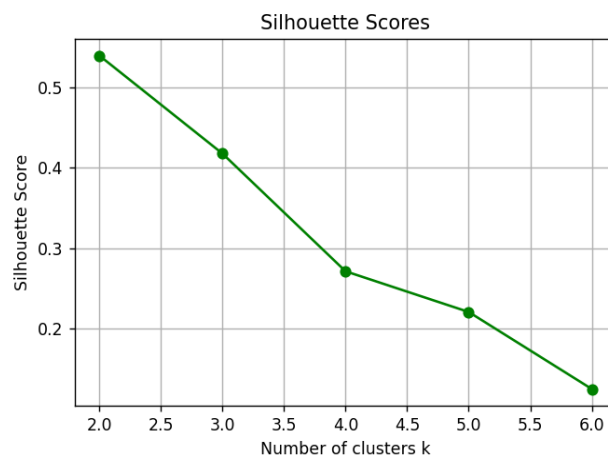
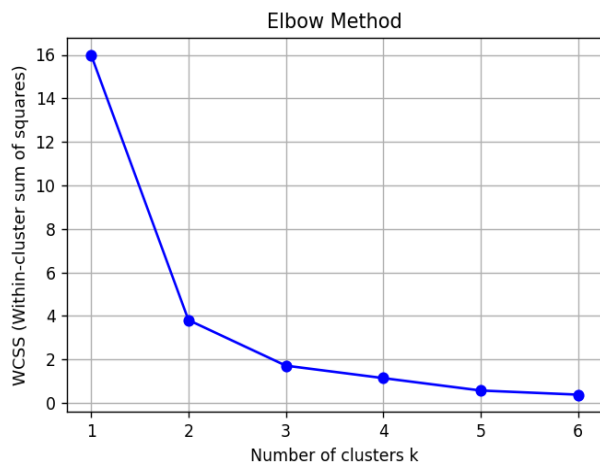
1. Prepare the data and visualize it in 2D before applying K-Means. Why is feature scaling important before clustering?





Name: SAKTHI.R
Cluster Labels: [1 0 0 1 1 0 0 1]

2. Plot the elbow curve and silhouette scores to identify the optimal number of clusters. Which value of k do you recommend and why?



Name: SAKTHI.R
Silhouette Score for $k=2$: 0.5398
Silhouette Score for $k=3$: 0.4179
Silhouette Score for $k=4$: 0.2714
Silhouette Score for $k=5$: 0.2208
Silhouette Score for $k=6$: 0.1250

3. Interpret each cluster and explain what kind of marketing strategy might work for that segment.

```
===== RESTART
Name: SAKTHI.R
Cluster 0 samples:
[[ 3 300]
 [ 4 400]
 [ 1 100]
 [ 2 200]]

Cluster 1 samples:
[[ 5 500]
 [ 6 600]
 [ 7 700]
 [ 8 800]]

Cluster 0 Average Feature Values: [ 2.5 250. ]
Cluster 1 Average Feature Values: [ 6.5 650. ]
>
```

Viva Questions:

1. What is clustering?
2. What is K-Means algorithm?
3. Is K-Means supervised or unsupervised?
4. What does the value of K represent?
5. How are centroids initialized?
6. What is Euclidean distance?
7. What is the elbow method?
8. What are limitations of K-Means?
9. How does K-Means handle large datasets?
10. Where is clustering used in engineering?

EX.NO:9	Time-series trend analysis
DATE:	

AIM:

To analyze and visualize time-series data.

ALGORITHM:

1. Load date-wise data.
2. Set datetime index and resample.
3. Plot trends with matplotlib.

PROGRAM: CODE

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

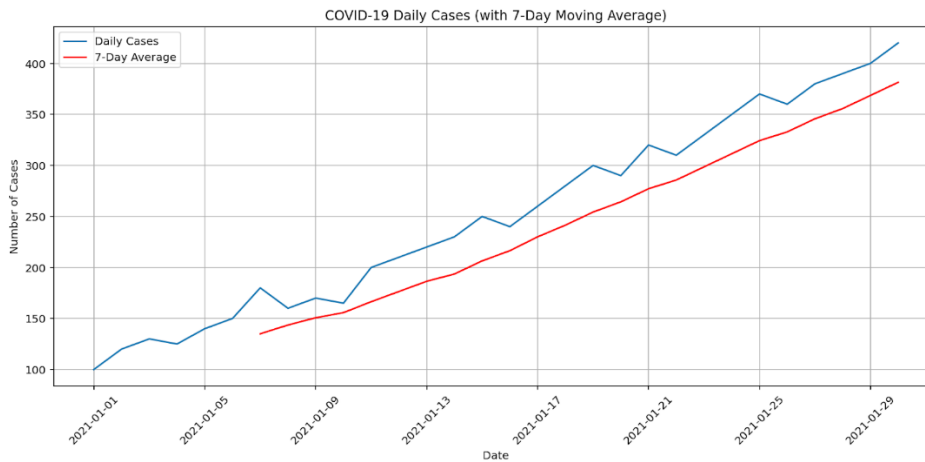
# Sample COVID-19 time-series dataset (simulated)
data = {
    'Date': pd.date_range(start='2021-01-01', periods=30,
                           freq='D'),
    'Daily_Cases': [ 100, 120, 130, 125, 140, 150, 180, 160, 170, 165, 200, 210, 220, 230, 250, 240, 260, 280, 300, 290, 320, 310, 330, 350, 370, 360, 380, 390, 400, 420 ]
}

df = pd.DataFrame(data)
df['7_day_avg'] = df['Daily_Cases'].rolling(window=7).mean()

# 7-day moving average

# Plotting
plt.figure(figsize=(12, 6))
sns.lineplot(x='Date', y='Daily_Cases', data=df, label='Daily Cases')
sns.lineplot(x='Date', y='7_day_avg', data=df, label='7-Day Average', color='red')
plt.title('COVID-19 Daily Cases (with 7-Day Moving Average)')
plt.xlabel('Date')
plt.ylabel('Number of Cases')
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

OUTPUT

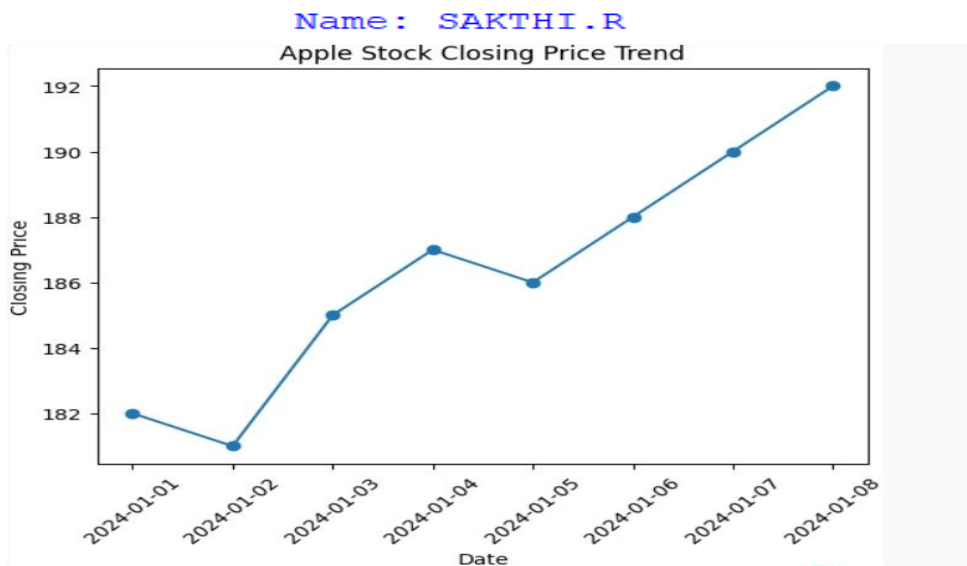


RESULT:

Thus, python program is successfully implemented to analyze and visualize time-series trends from datasets such as COVID-19 case statistics.

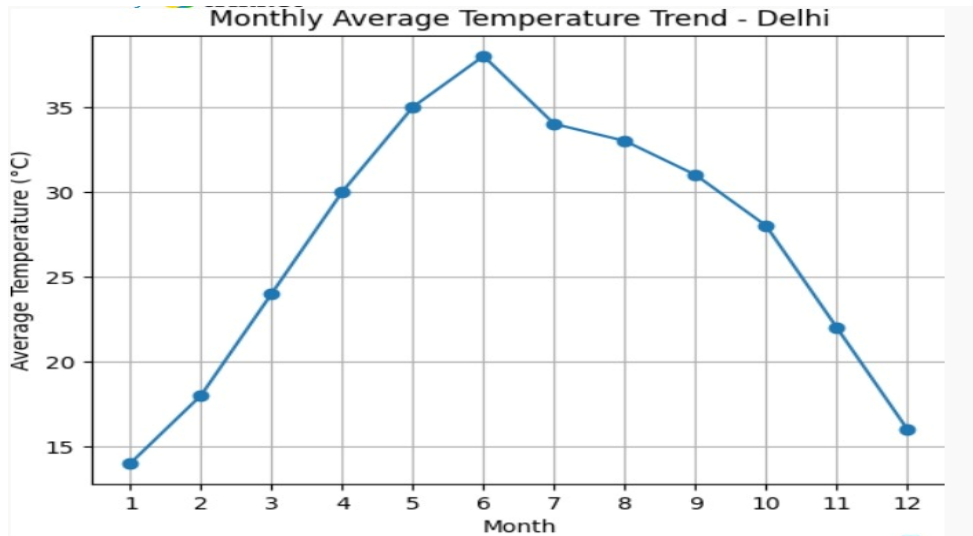
EXERCISE:

1. Load a CSV file named `stock_prices.csv` containing columns like Date, Company, Open, Close, High, Low, Volume. Filter the data for a specific company (e.g., "Apple"), convert Date to datetime, and plot the closing price trend over time.



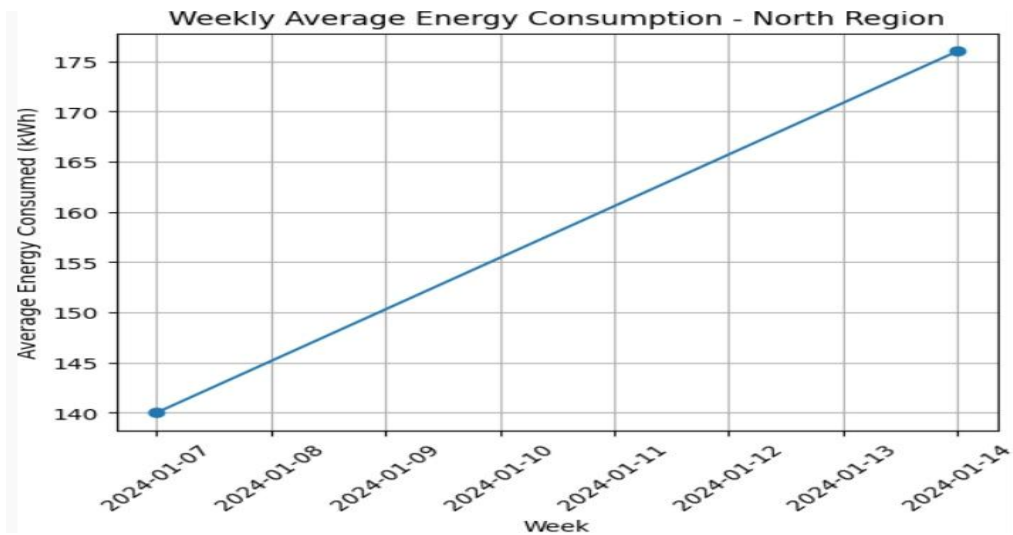
2. Given a dataset `weather_data.csv` with columns Date, City, Temperature, Humidity, Rainfall, filter the data for a city (e.g., "Delhi"), and plot the monthly average temperature trend over a year using a line chart.

Name : SAKTHI . R



3 .Load a time-series dataset energy_consumption.csv with Timestamp, Region, Energy_Consumed(kWh).Convert the timestamp, resample the data weekly, and plot the weekly average energy consumption for a region.

Name : SAKTHI . R



Viva Questions:

1. What is time-series data?
2. What are the components of time-series?
3. What is trend?
4. What is seasonality?
5. What is cyclic variation?
6. What is random noise?
7. How do you visualize time-series data?
8. What is moving average?
9. Why is trend analysis important?
10. Where is time-series analysis applied?

EX.No:10	Mini project: Statistical analysis of real-world data.
DATE:	

AIM:

To perform **statistical analysis of real-world data using Python** in order to compute descriptive statistics, analyze relationships between variables, and visualize data for meaningful insights.

ALGORITHM:

1. Start the program and import required libraries such as NumPy, Pandas, and Matplotlib.
2. Create or load a real-world dataset into a DataFrame.
3. Compute descriptive statistics such as mean, variance, and standard deviation.
4. Perform correlation analysis to study relationships between variables. 5. Visualize the data using appropriate plots and display the results.

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Step 1: Create a Real-World Dataset
# Example: Monthly Sales Data
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [12000, 15000, 17000, 16000, 18000, 20000],
    "Profit": [3000, 4200, 5000, 4800, 5500, 6200]
}
```

```
df = pd.DataFrame(data)
print("Dataset:\n", df)
```

```
# Step 2: Descriptive Statistics
print("\nStatistical Summary:\n")
print(df.describe())
```

```
# Mean, Variance, Standard Deviation
```

```
print("\nMean:\n", df.mean(numeric_only=True))
print("\nVariance:\n", df.var(numeric_only=True))
print("\nStandard Deviation:\n", df.std(numeric_only=True))
```

```

# Step 3: Correlation Analysis
correlation = df[["Sales", "Profit"]].corr()
print("\nCorrelation Matrix:\n", correlation)

# Step 4: Data Visualization
# Sales Trend
plt.figure()
plt.plot(df["Month"], df["Sales"])
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()

# Profit Trend
plt.figure()
plt.plot(df["Month"], df["Profit"])
plt.title("Monthly Profit Trend")
plt.xlabel("Month")
plt.ylabel("Profit")

plt.show()

# Scatter Plot (Sales vs Profit)
plt.figure()
plt.scatter(df["Sales"], df["Profit"])
plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")

plt.show()

# Step 5: Conclusion from Analysis
print("\nConclusion:")
print("The data shows an increasing trend in sales and profit.")
print("A strong positive correlation exists between sales and profit.")

```

OUTPUT:

```

In [17]: %runfile D:/SKCET/ASP/untitled6.py --wdir
Dataset:

```

	Month	Sales	Profit
0	Jan	12000	3000
1	Feb	15000	4200
2	Mar	17000	5000
3	Apr	16000	4800
4	May	18000	5500
5	Jun	20000	6200

```

Statistical Summary:

```

	Sales	Profit
count	6.000000	6.000000
mean	16333.333333	4783.333333
std	2732.520204	1103.479346
min	12000.000000	3000.000000
25%	15250.000000	4350.000000
50%	16500.000000	4900.000000
75%	17750.000000	5375.000000
max	20000.000000	6200.000000

```
Mean:
Sales      16333.333333
Profit     4783.333333
dtype: float64

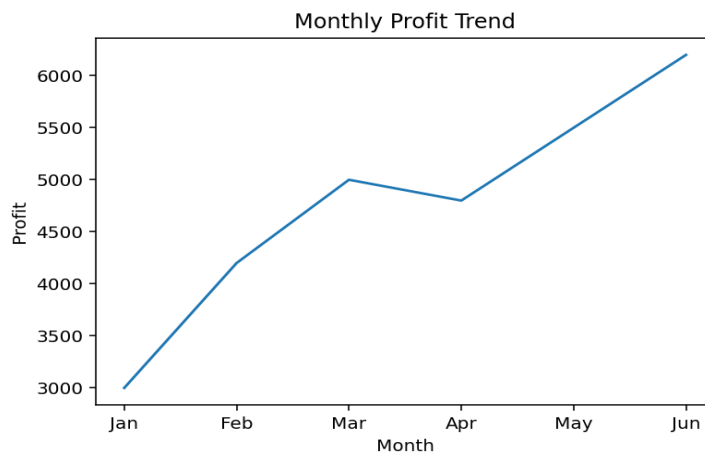
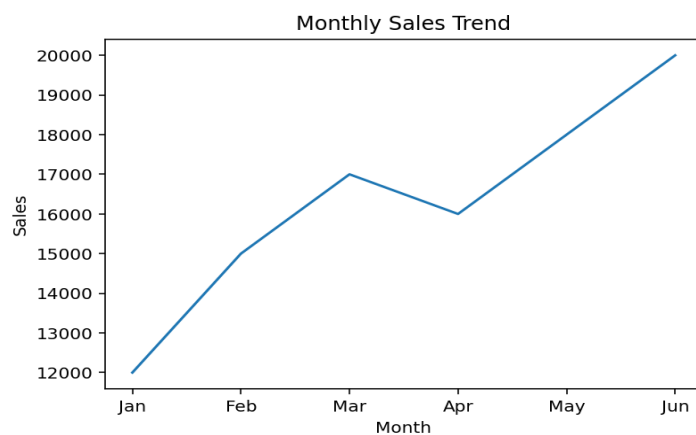
Variance:
Sales      7.466667e+06
Profit     1.217667e+06
dtype: float64

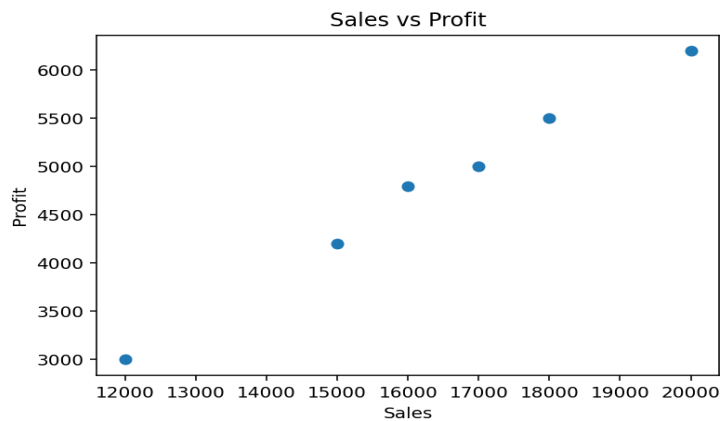
Standard Deviation:
Sales      2732.520204
Profit     1103.479346
dtype: float64

Correlation Matrix:
           Sales    Profit
Sales    1.000000  0.997144
Profit   0.997144  1.000000
```

Conclusion:

The data shows an increasing trend in sales and profit.
A strong positive correlation exists between sales and profit.





RESULT:

The program successfully analyzes the given real-world dataset and computes key statistical measures. It also visualizes trends and relationships, helping to derive meaningful insights from the data.

Exercise :

1. Write a Python program to perform statistical analysis on real-world weather data (temperature and rainfall).

```
===== RESTART: C:/Users
Name: SAKTHI.R
===== Weather Data =====
Day   Temperature   Rainfall
Mon           32           5
Tue           34           0
Wed           33          10
Thu           35           2
Fri           36           0
Sat           34           8
Sun           33           4

===== Temperature Analysis =====
Mean: 33.86
Median: 34.0
Max: 36
Min: 32
Standard Deviation: 1.25

===== Rainfall Analysis =====
Total Rainfall: 31
Mean Rainfall: 4.43
Max Rainfall: 10
Min Rainfall: 0

===== Correlation =====
Temperature vs Rainfall: -0.62
```

2. Modify the program to read data from a CSV file and compute descriptive statistics.

===== RESTART: C:/Users/sakth/AppData/Local/P

Name : SAKTHI.R

Weather Dataset:					
	Date	City	Temperature	Humidity	Rainfall
0	2024-01-10	Delhi	14	60	5
1	2024-02-12	Delhi	18	55	3
2	2024-03-15	Delhi	24	50	2
3	2024-04-18	Delhi	30	45	1
4	2024-05-20	Delhi	35	40	0

Descriptive Statistics:		
	Temperature	Rainfall
count	12.000000	12.000000
mean	26.916667	32.166667
std	7.982462	51.121661
min	14.000000	0.000000
25%	21.000000	2.750000
50%	29.000000	7.500000
75%	33.250000	30.000000
max	38.000000	150.000000

3. Extend the mini project to include regression or ANOVA analysis for deeper data interpretation.

===== RESTART: C:/Users/sakth/AppData/Local/P

Name : SAKTHI.R

Regression Results

OLS Regression Results

Dep. Variable:	Temperature	R-squared:	0.187
Model:	OLS	Adj. R-squared:	0.106
Method:	Least Squares	F-statistic:	2.303
Date:	Mon, 16 Mar 2026	Prob (F-statistic):	0.160
Time:	19:05:58	Log-Likelihood:	-40.189
No. Observations:	12	AIC:	84.38
Df Residuals:	10	BIC:	85.35
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	24.7435	2.607	9.490	0.000	18.934	30.553
Rainfall	0.0676	0.045	1.518	0.160	-0.032	0.167

Omnibus:	0.165	Durbin-Watson:	0.595
Prob(Omnibus):	0.921	Jarque-Bera (JB):	0.341
Skew:	0.188	Prob(JB):	0.843
Kurtosis:	2.264	Cond. No.	70.1

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

ANOVA Table

	sum_sq	df	F	PR(>F)
C(Season)	576.250000	3.0	12.326203	0.002281
Residual	124.666667	8.0	NaN	NaN

Viva Questions:

1. What was the objective of your mini project?
2. What type of data did you use?
3. How did you collect the dataset?
4. What preprocessing steps were applied?
5. How did you handle missing values?
6. Which statistical techniques were used?
7. What challenges did you face?
8. How did visualization help your analysis?
9. What conclusions were drawn from the data?
10. How can your project be improved further?